



Tabla de contenido

QUERYBUILDER.....	2
OPERADORES PARA APLICAR EN FILTROS DE CONSULTAS	2
COMPARACION	2
LOGICOS.....	2
DE ELEMENTOS.....	3
DE EVALUACION	3
DE ARRAYS	4
EJEMPLOS	4
SCHEMES COMPLEJOS	6



QUERYBUILDER

El QueryBuilder en Mongoose sirve para construir consultas a la base de datos MongoDB de forma encadenada y progresiva, permitiendo crear consultas complejas de manera más sencilla y legible. Mediante métodos encadenados se puede ir agregando filtros, ordenar, limitar registros, seleccionar campos específicos y finalmente ejecutar la consulta. Es una abstracción que facilita el manejo de operaciones CRUD y patrones habituales de consulta, integrándose naturalmente con la API de Mongoose.

- Facilita la construcción y ejecución de consultas complejas.
- Permite encadenar operaciones como filtros, límites, ordenamientos y selecciones.
- Puede incluir métodos personalizados para patrones comunes de consulta.
- Mejora la legibilidad y mantenimiento del código que interactúa con MongoDB mediante Mongoose.

OPERADORES PARA APLICAR EN FILTROS DE CONSULTAS

COMPARACION

Operador	Uso	Ejemplo
<code>\$eq</code>	Igual a un valor	<code>{ edad: { \$eq: 25 } }</code>
<code>\$ne</code>	Distinto de un valor	<code>{ estado: { \$ne: "activo" } }</code>
<code>\$gt</code>	Mayor que	<code>{ edad: { \$gt: 18 } }</code>
<code>\$gte</code>	Mayor o igual que	<code>{ edad: { \$gte: 18 } }</code>
<code>\$lt</code>	Menor que	<code>{ edad: { \$lt: 65 } }</code>
<code>\$lte</code>	Menor o igual que	<code>{ edad: { \$lte: 65 } }</code>
<code>\$in</code>	Coincide con algún valor de la lista	<code>{ pais: { \$in: ["Colombia", "México"] } }</code>
<code>\$nin</code>	No coincide con ninguno de la lista	<code>{ pais: { \$nin: ["Perú", "Chile"] } }</code>

LOGICOS

Operador	Uso	Ejemplo
----------	-----	---------



\$and	Todas las condiciones deben cumplirse	{ \$and: [{ edad: { \$gte: 18 } }, { activo: true }] }
\$or	Alguna condición debe cumplirse	{ \$or: [{ rol: "admin" }, { rol: "editor" }] }
\$not	Niega la condición	{ edad: { \$not: { \$gt: 30 } } }
\$nor	Ninguna de las condiciones debe cumplirse	{ \$nor: [{ rol: "admin" }, { activo: false }] }

DE ELEMENTOS

Operador	Uso	Ejemplo
\$exists	Verifica si un campo existe	{ telefono: { \$exists: true } }
\$type	Filtra por tipo de dato	{ edad: { \$type: "number" } }

DE EVALUACION

Operador	Uso	Ejemplo
\$regex	Expresión regular (coincidencia de patrones)	{ nombre: { \$regex: /^A/, \$options: "i" } }
\$expr	Usar expresiones dentro de la consulta	{ \$expr: { \$gt: ["\$ventas", "\$compras"] } }
\$jsonSchema	Valida contra un esquema JSON	{ \$jsonSchema: { required: ["nombre", "edad"] } }
\$mod	Valor módulo de un número	{ edad: { \$mod: [2, 0] } } // pares
\$text	Búsqueda de texto	{ \$text: { \$search: "mongoose" } }
\$where	Expresión JavaScript personalizada	{ \$where: "this.edad > this.aniosExperiencia" }



DE ARRAYS

Operador	Uso	Ejemplo
\$all	Contiene todos los elementos	{ tags: { \$all: ["node", "js"] }}
\$elemMatch	Coincide con un elemento del array	{ notas: { \$elemMatch: { materia: "math", nota: { \$gte: 4 } } } }
\$size	Tamaño del array	{ amigos: { \$size: 3 } }

EJEMPLOS

1. Buscar documentos con filtro simple

Filtrar documentos donde el campo status sea "activo":

```
Model.find()
  .where('status').equals('activo')
  .exec((err, resultados) => {
    if (err) return handleError(err);
    console.log(resultados);
  });
```

2. Buscar documentos con condiciones múltiples

Filtrar donde status sea "activo" y age mayor que 18:

```
Model.find()
  .where('status').equals('activo')
  .where('age').gt(18)
```



```
.exec((err, resultados) => {  
  console.log(resultados);  
});
```

3. Ordenar resultados

Ordenar los documentos por createdAt descendente:

```
Model.find()  
  .sort({ createdAt: -1 })  
  .exec((err, resultados) => {  
    console.log(resultados);  
  });
```

4. Seleccionar solo campos específicos

Obtener solo los campos name y email:

```
Model.find()  
  .select('name email')  
  .exec((err, resultados) => {  
    console.log(resultados);  
  });
```

5. Combinar todo en una consulta

```
Model.find()  
  .where('status').equals('activo')  
  .where('age').gt(18)  
  .sort({ createdAt: -1 })  
  .skip(10)  
  .limit(5)  
  .select('name email age')
```



```
.exec((err, resultados) => {  
  console.log(resultados);  
});
```

SCHEMES COMPLEJOS

MongoDB no es una base de datos relacional, por lo que no hay comandos JOIN al estilo SQL. En su lugar, se usan:

- población (populate)
- subdocumentos

MongoDB y Mongoose no admiten comandos JOIN, ya que MongoDB no es una base de datos relacional. Mongoose le ofrece una forma elegante de lograr una Funcionalidad similar mediante el uso de POPULATE.

populate funciona extrayendo información del segundo modelo al modelo que estás utilizando actualmente.

A diferencia de las sentencias JOIN, **en realidad se trata de consultas independientes**. Primero recuperas tu conjunto principal de datos de una colección y, una vez que se devuelve, «poblas» los datos necesarios desde una colección secundaria.